

HotelManager

Documentação de Produto

Sistema Gerencial para Hotel e Pousada

Tecnologia principal: Java com Spring Boot

Banco de dados sugerido: MySQL

Tipo de sistema: API REST com arquitetura em camadas

Finalidade: Gerenciar quartos, hóspedes, reservas, hospedagens, check-in, check-out, consumo, pagamentos, limpeza, manutenção e indicadores administrativos de um estabelecimento hoteleiro.

00 Sumário

01 - Introdução e Escopo	03
02 - Objetivo do Sistema	04
03 - Público-Alvo e Atores	05
04 - Funcionalidades Principais	06
05 - Regras de Negócio	12
06 - Entidades do Sistema	14
07 - Relacionamentos entre Entidades	17
08 - Estrutura do Projeto Spring Boot	18
09 - Endpoints REST Sugeridos	20
10 - Tecnologias Sugeridas	23
11 - Banco de Dados	24

12 - Fluxo Básico do Sistema	25
13 - Requisitos Funcionais	26
14 - Requisitos Não Funcionais	29
15 - Considerações Finais	31

01 Introdução e Escopo

Introdução

O projeto **HotelManager** tem como objetivo desenvolver um sistema para gerenciamento de hotel, pousada ou pequeno estabelecimento de hospedagem, substituindo controles manuais, planilhas dispersas e registros informais por uma solução digital organizada, segura e de fácil utilização.

Em um ambiente hoteleiro, informações como disponibilidade de quartos, reservas, check-ins, check-outs, pagamentos, consumos e limpeza precisam estar sempre atualizadas. Quando esses dados são controlados manualmente, há maior risco de conflito de reservas, cobrança incorreta, perda de histórico do hóspede e dificuldade na tomada de decisões administrativas.

O HotelManager propõe centralizar essas operações em um sistema desenvolvido em **Java com Spring Boot**, utilizando banco de dados relacional para armazenar as informações principais do hotel e disponibilizando uma **API REST** para comunicação com futuras interfaces web, mobile ou integrações externas.

Escopo do Projeto

O sistema será responsável por controlar as principais rotinas operacionais e administrativas de um hotel, incluindo cadastro de quartos, cadastro de hóspedes, reservas, hospedagens, check-in, check-out, serviços adicionais, pagamentos, limpeza, manutenção e relatórios administrativos.

Nesta primeira versão, o sistema não terá como foco integrações com plataformas externas de reserva, emissão de nota fiscal, integração bancária automática, controle de fechaduras eletrônicas, aplicativo mobile nativo ou motor público de reservas online.

02 Objetivo do Sistema

Objetivo Geral

Desenvolver um sistema completo para gerenciamento de hotel ou pousada, permitindo o controle centralizado das operações de hospedagem, reservas, quartos, hóspedes, pagamentos, consumos, limpeza, manutenção e indicadores administrativos.

Objetivos Específicos

- Permitir o cadastro e gerenciamento de quartos.
- Controlar o status dos quartos em tempo real.
- Permitir o cadastro de hóspedes.
- Registrar reservas com datas de entrada e saída.
- Evitar conflitos de reserva para o mesmo quarto.
- Calcular automaticamente a quantidade de diárias.
- Calcular valores previstos e valores finais de hospedagem.
- Permitir check-in com ou sem reserva.
- Permitir check-out com fechamento financeiro.
- Registrar consumos e serviços adicionais.
- Registrar pagamentos totais ou parciais.
- Controlar limpeza e manutenção de quartos.
- Exibir relatórios e indicadores administrativos.
- Organizar o desenvolvimento em camadas usando Spring Boot.

03 Público-Alvo e Atores

Público-Alvo

O HotelManager é destinado a hotéis de pequeno e médio porte, pousadas, hostels, estabelecimentos familiares de hospedagem, funcionários de recepção, administradores, gerentes, equipes de limpeza, equipes de manutenção e responsáveis financeiros.

Tabela de Atores

Tipo de Usuário	Papel no Sistema	Permissões	Restrições
Administrador/ Gerente	Responsável pela gestão geral do hotel.	Cadastrar quartos, visualizar relatórios, acompanhar receita, gerenciar reservas, hospedagens e pagamentos.	Deve possuir acesso restrito e controlado.
Recepcionista	Responsável pelo atendimento ao hóspede.	Cadastrar hóspedes, criar reservas, realizar check-in, realizar check-out e consultar quartos disponíveis.	Não deve alterar configurações administrativas críticas.
Equipe de Limpeza	Responsável pela atualização do estado dos quartos.	Visualizar quartos sujos, marcar quarto em limpeza e finalizar limpeza.	Não deve acessar dados financeiros.
Equipe de Manutenção	Responsável por quartos indisponíveis por reparo.	Registrar manutenção, informar motivo e liberar quarto após conclusão.	Não deve realizar reservas ou pagamentos.
Financeiro	Responsável pelo controle de pagamentos.	Consultar hospedagens, registrar pagamentos e acompanhar status financeiro.	Não deve alterar dados operacionais sem autorização.

04 Funcionalidades Principais

Módulo de Quartos

O módulo de quartos será responsável por controlar o cadastro e a situação operacional dos quartos do hotel.

- Cadastrar, editar, excluir ou inativar quarto.
- Definir número do quarto, tipo, capacidade, valor da diária, descrição e status.
- Controlar se o quarto está ativo ou inativo.
- Alterar status do quarto conforme operação do hotel.

Tipos sugeridos: solteiro, casal, luxo, suíte, família, executivo e presidencial.

Status possíveis: disponível, ocupado, reservado, sujo, em limpeza e em manutenção.

Módulo de Hóspedes

O módulo de hóspedes permitirá manter os dados das pessoas que utilizam os serviços do hotel.

- Cadastrar hóspede.
- Editar dados do hóspede.
- Excluir ou inativar cadastro.
- Consultar histórico de hospedagens.
- Consultar reservas vinculadas.

Dados principais: nome, CPF ou documento, telefone, e-mail, endereço, data de cadastro e observações.

04 Funcionalidades Principais

Módulo de Reservas

O módulo de reservas permitirá registrar hospedagens futuras, garantindo controle de disponibilidade e evitando conflitos.

- Criar reserva.
- Selecionar hóspede e quarto.
- Definir data de entrada e data de saída.
- Calcular quantidade de diárias.
- Calcular valor previsto.
- Alterar, cancelar e finalizar reserva.
- Registrar não comparecimento.

Status possíveis: ativa, cancelada, finalizada e não compareceu.

Módulo de Check-in

O módulo de check-in será utilizado para confirmar a entrada do hóspede no hotel.

- Confirmar entrada do hóspede.
- Validar se existe reserva ativa.
- Permitir check-in direto quando houver quarto disponível.
- Registrar data e hora de entrada.
- Vincular hóspede ao quarto.
- Criar hospedagem em andamento.
- Alterar status do quarto para ocupado.

04 Funcionalidades Principais

Módulo de Check-out

O módulo de check-out será responsável por finalizar a hospedagem e calcular os valores devidos.

- Finalizar hospedagem.
- Registrar data e hora de saída.
- Calcular valor das diárias.
- Somar consumos e serviços adicionais.
- Calcular valor total da hospedagem.
- Registrar pagamento.
- Definir status financeiro.
- Alterar quarto para sujo após a saída.

Módulo de Limpeza

- Listar quartos sujos.
- Marcar quarto como em limpeza.
- Registrar início da limpeza.
- Registrar responsável pela limpeza.
- Finalizar limpeza.
- Alterar quarto para disponível após limpeza.
- Consultar histórico de limpezas.

Status possíveis: pendente, em andamento e finalizada.

04 Funcionalidades Principais

Módulo de Manutenção

- Marcar quarto como em manutenção.
- Informar motivo da manutenção.
- Registrar data de início e data de fim.
- Bloquear quarto para novas reservas.
- Finalizar manutenção.
- Liberar quarto após manutenção.
- Consultar histórico de manutenções.

Status possíveis: aberta, em andamento e finalizada.

Módulo Financeiro

- Registrar pagamento.
- Consultar pagamentos.
- Associar pagamento a uma hospedagem.
- Controlar pagamentos pendentes, parciais e quitados.
- Calcular valor total da hospedagem.
- Registrar forma de pagamento.

Formas de pagamento: dinheiro, cartão, Pix e outro.

Status do pagamento: pendente, pago e parcial.

04 Funcionalidades Principais

Módulo de Serviços e Consumo

Esse módulo permitirá registrar serviços adicionais utilizados pelo hóspede durante a hospedagem.

- Cadastrar serviços adicionais.
- Editar serviços.
- Inativar serviços.
- Vincular consumo à hospedagem.
- Informar quantidade consumida.
- Calcular valor total do consumo.
- Somar consumos ao check-out.

Serviços sugeridos: frigobar, lavanderia, café da manhã, estacionamento, serviço de quarto e consumos diversos.

Módulo de Dashboard

O dashboard apresentará uma visão resumida da situação operacional e financeira do hotel.

- Total de quartos.
- Quartos disponíveis, ocupados, reservados, sujos, em limpeza e em manutenção.
- Reservas do dia.
- Check-ins do dia.
- Check-outs do dia.
- Receita total.
- Taxa de ocupação.

Tabela de Regras de Negócio

ID	Regra
RN01	Um quarto ocupado não pode ser reservado por outro hóspede.
RN02	Um quarto em manutenção não pode ser reservado.
RN03	Um quarto sujo não deve aparecer como disponível.
RN04	Ao realizar check-out, o quarto deve mudar automaticamente para o status sujo.
RN05	Após a finalização da limpeza, o quarto pode voltar para disponível.
RN06	A reserva deve calcular automaticamente o valor previsto com base na quantidade de diárias.
RN07	O check-in só pode ser realizado se houver uma reserva válida ou um quarto disponível.
RN08	O check-out deve calcular diárias, consumos e serviços adicionais.
RN09	O sistema deve impedir conflito de datas para o mesmo quarto.
RN10	O hóspede precisa estar cadastrado antes de uma reserva ou check-in.
RN11	Uma reserva cancelada não pode ser utilizada para check-in.
RN12	Uma hospedagem finalizada não pode receber novos consumos.
RN13	Um pagamento não pode ultrapassar o valor total da hospedagem, salvo regra administrativa futura.
RN14	Um quarto inativo não deve aparecer em reservas ou check-ins.
RN15	O status de pagamento deve ser atualizado conforme os valores pagos.
RN16	A diária deve considerar a diferença entre data de entrada e data de saída.
RN17	Uma manutenção aberta deve bloquear automaticamente o quarto.
RN18	Um quarto em limpeza não deve ser selecionado para nova hospedagem.

06 Entidades do Sistema

Classes Java Sugeridas

Entidade	Atributos
Quarto	id, numero, tipo, capacidade, valorDiaria, status, descricao, ativo

Hospede	id, nome, documento, telefone, email, endereco, dataCadastro, observacoes
Reserva	id, hospede, quarto, dataEntrada, dataSaida, quantidadeDiarias, status, valorPrevisto, dataCriacao
Hospedagem	id, hospede, quarto, reserva, dataCheckin, dataCheckout, valorDiarias, valorConsumos, valorTotal, status
Pagamento	id, hospedagem, valor, formaPagamento, status, dataPagamento, observacao
Servico	id, nome, descricao, valor, ativo
Consumo	id, hospedagem, servico, quantidade, valorUnitario, valorTotal, dataConsumo
Manutencao	id, quarto, descricao, dataInicio, dataFim, responsavel, status
Limpeza	id, quarto, dataInicio, dataFim, responsavel, status, observacao

07Relacionamentos entre Entidades

Relacionamentos Esperados

- Um hóspede pode possuir várias reservas.
- Uma reserva pertence a um único hóspede.
- Um quarto pode possuir várias reservas em datas diferentes.
- Uma reserva pertence a um único quarto.
- Uma hospedagem pertence a um hóspede.
- Uma hospedagem pertence a um quarto.
- Uma hospedagem pode ser originada de uma reserva.
- Uma hospedagem pode possuir vários consumos.
- Um consumo pertence a uma hospedagem.
- Um consumo está vinculado a um serviço.
- Uma hospedagem pode possuir um ou mais pagamentos.
- Um pagamento pertence a uma hospedagem.
- Um quarto pode possuir vários registros de limpeza.

- Uma limpeza pertence a um quarto.
- Um quarto pode possuir vários registros de manutenção.
- Uma manutenção pertence a um quarto.

08 Estrutura do Projeto Spring Boot

Pacotes Sugeridos

<code>com.hotelmanager</code>	<code>com.hotelmanager.model</code>	<code>com.hotelmanager.repository</code>
<code>com.hotelmanager.service</code>	<code>com.hotelmanager.controller</code>	
<code>com.hotelmanager.dto</code>	<code>com.hotelmanager.enums</code>	<code>com.hotelmanager.exception</code>

Responsabilidade das Camadas

Camada	Responsabilidade
model/entity	Representar as entidades do banco de dados usando classes Java e anotações JPA.
repository	Realizar comunicação com o banco de dados por meio do Spring Data JPA.
service	Concentrar regras de negócio, validações e processamento das operações.
controller	Disponibilizar endpoints REST para entrada e saída de dados.
dto	Transportar dados entre cliente e servidor sem expor diretamente as entidades.
enums	Definir valores fixos como status de quarto, status de reserva e forma de pagamento.
exception	Centralizar exceções e tratamentos de erro do sistema.

Enums sugeridos: TipoQuarto, StatusQuarto, StatusReserva, StatusHospedagem, FormaPagamento, StatusPagamento, StatusLimpeza e StatusManutencao.

09 Endpoints REST Sugeridos

Endpoints por Módulo

Módulo	Endpoints
Quartos	GET /quartos
	GET /quartos/{id}
	GET /quartos/disponiveis
	POST /quartos
	PUT /quartos/{id}
	DELETE /quartos/{id}
	PATCH /quartos/{id}/status
Hóspedes	GET /hospedes
	GET /hospedes/{id}
	GET /hospedes/{id}/historico
	POST /hospedes
	PUT /hospedes/{id}
	DELETE /hospedes/{id}
Reservas	GET /reservas
	GET /reservas/{id}
	GET /reservas/data
	POST /reservas
	PUT /reservas/{id}
	PATCH /reservas/{id}/cancelar
	PATCH /reservas/{id}/finalizar
	PATCH /reservas/{id}/nao-compareceu
	POST /checkin
Check-in	POST /checkin/reserva/{reservaid}
	POST /checkin/direto
Check-out	POST /checkout
	POST /checkout/hospedagem/{hospedagemId}

09 Endpoints REST Sugeridos

Módulo	Endpoints
Limpeza	GET /limpezas
	GET /limpezas/quartos-sujos
	POST /limpezas
	PATCH /limpezas/{id}/iniciar
	PATCH /limpezas/{id}/finalizar
Manutenção	GET /manutencoes
	POST /manutencoes
	PATCH /manutencoes/{id}/iniciar
	PATCH /manutencoes/{id}/finalizar
Serviços e Consumos	GET /servicos
	POST /servicos
	PUT /servicos/{id}
	DELETE /servicos/{id}
	GET /consumos
	POST /consumos
Financeiro	GET /hospedagens/{id}/consumos
	GET /pagamentos
	GET /pagamentos/{id}
	POST /pagamentos
	GET /hospedagens/{id}/pagamentos
Dashboard	GET /dashboard/resumo
	GET /dashboard/ocupacao
	GET /dashboard/receita

10 Tecnologias Sugeridas

Tabela de Tecnologias

Tecnologia	Finalidade
Java	Linguagem principal do sistema.
Spring Boot	Framework para criação da aplicação backend.
Spring Web	Criação de endpoints REST.
Spring Data JPA	Comunicação com o banco de dados.

Hibernate	Implementação ORM para persistência das entidades.
MySQL	Banco de dados relacional sugerido.
Maven	Gerenciamento de dependências e build do projeto.
API REST	Comunicação entre backend e futuras interfaces.
Insomnia/Postman	Testes dos endpoints.
Git/GitHub	Versionamento do código-fonte.

11 Banco de Dados

Banco Relacional

O HotelManager deverá utilizar banco de dados relacional, preferencialmente **MySQL**, pois o sistema possui dados estruturados e relacionamentos claros entre quartos, hóspedes, reservas, hospedagens, pagamentos, consumos, limpezas e manutenções.

Tabelas Principais

- quartos
- hospedes
- reservas
- hospedagens
- pagamentos
- servicos
- consumos
- limpezas
- manutencoes

Observações

- As tabelas devem possuir chave primária id.
- Relacionamentos devem ser representados por chaves estrangeiras.
- Campos de status devem preferencialmente usar enums no Java.
- Exclusões importantes podem ser feitas por inativação lógica.
- Datas de check-in, check-out, limpeza e manutenção devem ser registradas para histórico.
- Valores monetários devem utilizar tipo adequado, como BigDecimal no Java.

12 Fluxo Básico do Sistema

Fluxo Prático de Uso

1. O administrador cadastra os quartos do hotel.
2. A recepção cadastra o hóspede.
3. A recepção cria uma reserva ou realiza check-in direto.
4. O sistema valida a disponibilidade do quarto.
5. O sistema altera o status do quarto para ocupado.
6. Durante a hospedagem, consumos e serviços podem ser adicionados.
7. No check-out, o sistema calcula o valor das diárias.
8. O sistema soma consumos e serviços adicionais.
9. O pagamento é registrado.
10. A hospedagem é finalizada.
11. O quarto muda automaticamente para sujo.
12. A equipe de limpeza inicia e finaliza a limpeza.
13. Após a limpeza, o quarto volta para disponível.
14. O gerente acompanha indicadores pelo dashboard.

13Requisitos Funcionais

Tabela de Requisitos Funcionais

ID	Descrição
RF01	O sistema deve permitir cadastrar quartos.
RF02	O sistema deve permitir editar quartos.
RF03	O sistema deve permitir inativar quartos.
RF04	O sistema deve controlar o status dos quartos.
RF05	O sistema deve permitir cadastrar hóspedes.
RF06	O sistema deve permitir editar hóspedes.
RF07	O sistema deve permitir consultar histórico de hospedagens do hóspede.
RF08	O sistema deve permitir criar reservas.
RF09	O sistema deve permitir cancelar reservas.
RF10	O sistema deve calcular automaticamente a quantidade de diárias.
RF11	O sistema deve calcular o valor previsto da reserva.
RF12	O sistema deve impedir reserva com conflito de datas.
RF13	O sistema deve permitir realizar check-in.
RF14	O sistema deve permitir realizar check-in direto em quarto disponível.
RF15	O sistema deve alterar o quarto para ocupado após check-in.

13Requisitos Funcionais

ID	Descrição
RF16	O sistema deve permitir realizar check-out.
RF17	O sistema deve calcular o valor total da hospedagem.
RF18	O sistema deve permitir registrar consumos.
RF19	O sistema deve permitir cadastrar serviços adicionais.
RF20	O sistema deve permitir registrar pagamentos.
RF21	O sistema deve controlar pagamento pendente, parcial e pago.
RF22	O sistema deve alterar o quarto para sujo após check-out.

- RF23 O sistema deve permitir registrar limpeza de quarto.
- RF24 O sistema deve permitir liberar quarto após limpeza.
- RF25 O sistema deve permitir registrar manutenção de quarto.
- RF26 O sistema deve bloquear quarto em manutenção para reservas.
- RF27 O sistema deve exibir dashboard administrativo.
- RF28 O sistema deve listar reservas do dia.
- RF29 O sistema deve listar check-ins e check-outs do dia.
- RF30 O sistema deve apresentar indicadores de ocupação e receita.

14Requisitos Não Funcionais

Tabela de Requisitos Não Funcionais

ID	Descrição
RNF01	O sistema deve ser desenvolvido em Java com Spring Boot.
RNF02	O sistema deve utilizar banco de dados relacional MySQL.
RNF03	O sistema deve disponibilizar API REST.
RNF04	O sistema deve seguir arquitetura em camadas.
RNF05	O sistema deve utilizar Spring Data JPA para persistência.
RNF06	O sistema deve utilizar Maven para gerenciamento de dependências.
RNF07	O sistema deve ser versionado com Git.
RNF08	O código deve ser limpo, organizado e padronizado.
RNF09	O sistema deve possuir tratamento centralizado de erros.
RNF10	O sistema deve validar dados obrigatórios antes de salvar registros.
RNF11	O sistema deve utilizar nomes claros para classes, métodos e pacotes.
RNF12	O sistema deve permitir testes dos endpoints por Insomnia ou Postman.
RNF13	O sistema deve armazenar valores monetários com precisão adequada.
RNF14	O sistema deve manter histórico de operações importantes.
RNF15	O sistema deve ser preparado para futuras integrações.

15 Considerações Finais

Conclusão

A documentação do **HotelManager** servirá como base para o desenvolvimento de um sistema completo de gerenciamento hoteleiro em Java com Spring Boot.

O projeto foi planejado para atender necessidades reais de hotéis, pousadas e pequenos estabelecimentos de hospedagem, centralizando rotinas essenciais como cadastro de quartos, controle de reservas, check-in, check-out, consumos, pagamentos, limpeza, manutenção e indicadores administrativos.

Com essa documentação, será possível iniciar a implementação do sistema de forma organizada, seguindo uma estrutura clara de entidades, regras de negócio, endpoints, banco de dados, requisitos funcionais e requisitos não funcionais.